

Two colored squares are positioned side-by-side at the top left. The left square is yellow and contains the text 'DOM' in bold black font. The right square is teal and contains the text 'Event' in bold white font.

DOM

Event

DOM Manipulation & Events

Mengontrol HTML & Merespons Interaksi Pengguna via JavaScript

Lanjutan: JavaScript & Node.js Fundamentals

DOM Tree → Selection → Manipulasi → Events → Praktik

DOM & Seleksi	Manipulasi	Events	Praktik
Apa itu DOM & DOM Tree	Baca & Ubah konten/atribut	addEventListener & jenis event	PRAKTIK 1 — Todo List
querySelector & querySelectorAll	Buat & hapus elemen	e.target & preventDefault	PRAKTIK 2 — Galeri + Filter
Traversal: parent, children, sibling	classList & style	Event Delegation	PRAKTIK 3 — Countdown Timer

SEKSI 1

DOM & Selection Elemen

Document Object Model — Tree objek HTML yang bisa dikontrol JavaScript

struktur halaman HTML yang dibaca browser menjadi bentuk seperti pohon (tree), sehingga JavaScript bisa mengakses dan mengubah isi halaman.

1.1 Detail | `querySelector`, `querySelectorAll` & Traversal

Penjelasan mendalam setiap method seleksi dan traversal DOM

`querySelector(selector)`

- Mencari elemen pertama yang cocok dengan CSS selector
- Mengembalikan null jika tidak ditemukan (bukan error)
- Lebih fleksibel dari `getElementById` karena menerima semua CSS selector: tag, .class, #id, [attr], :pseudo
- Hanya mengembalikan 1 elemen — elemen pertama yang cocok

`querySelectorAll(selector)`

- Mengembalikan `NodeList` — mirip array tapi BUKAN array sejati
- `NodeList` bersifat static — tidak auto-update jika DOM berubah
- Gunakan `Array.from(list)` atau `[...list]` agar bisa pakai `.map()`
- Jika tidak ada yang cocok → `NodeList` kosong (bukan null)

`getElementById(id)`

- Lebih cepat dari `querySelector` karena khusus mencari by ID
- Tidak perlu tanda # — langsung tulis nama ID-nya
- Kembalikan null jika ID tidak ada di halaman

Traversal — Gerak dalam Pohon/Tree DOM

- | | |
|---------------------------------------|---|
| • <code>parentElement</code> | → naik ke elemen induk (1 tingkat) |
| • <code>children</code> | → semua child element (bukan text node) |
| • <code>firstElementChild</code> | → child pertama |
| • <code>lastElementChild</code> | → child terakhir |
| • <code>nextElementSibling</code> | → saudara setelahnya (level sama) |
| • <code>previousElementSibling</code> | → saudara sebelumnya |

1.1 | DOM Tree & Seleksi Elemen

Setiap tag HTML menjadi node dalam tree — JS bisa mengakses semua node

DOM Tree

- `<html>` = root node
- ↳ `<head>` — `<body>`
- ↳ `<div>` ↳ `<p>` ↳ text
- Setiap elemen = objek dengan properti & method
- JS mengakses DOM lewat objek 'document'

```
// Selection 1 elemen (pertama cocok)
const el = document.querySelector('.card');
const h1 = document.querySelector('h1');
const byId = document.getElementById('app');

// Selection SEMUA elemen yang cocok
const all = document.querySelectorAll('li');
all.forEach(li => console.log(li.textContent));

// Traversal — gerak dalam pohon
el.parentElement // naik ke parent
el.children // semua child element
el.firstElementChild // child pertama
el.lastElementChild // child terakhir
el.nextElementSibling // saudara setelahnya
el.previousElementSibling // saudara sebelumnya

// Cek apakah cocok selector
el.matches('.active') // true / false
el.closest('.wrapper') // naik sampai ketemu
```

Selector Lengkap

- 'p' — tag element
- '.card' — class
- '#main' — id
- 'div > p' — child langsung
- 'input[type]' — attribute
- ':first-child' — pseudo-class

```
<div class="wrapper">
```

```
  <div class="card">
    <h1>Judul</h1>
    <p>Deskripsi</p>
    <button>Save</button>
  </div>
```

```
</div>
```

- `const card = document.querySelector('.card');`
 - `<div class="card">`
- `parentElement`
 - `card.parentElement`
 - `<div class="wrapper">`
- `children`
 - `card.children`
 - `[h1, p, button]`

1.2 | Read, Update, Create & Hapus Elemen

textContent vs innerHTML

textContent

- Membaca/menulis teks murni — tag HTML diabaikan
- Aman dari serangan XSS (Cross-Site Scripting)
- Gunakan ini jika konten berasal dari input user

innerHTML

- Membaca/menulis termasuk tag HTML di dalamnya
- Berbahaya jika isi berasal dari user — bisa disisipi script
- Gunakan hanya jika konten sudah aman/terkontrol

Atribut — `getAttribute` & `setAttribute`

- `getAttribute("href")` → baca nilai atribut
- `setAttribute("data-id", "5")` → ubah/tambah atribut
- `removeAttribute("disabled")` → hapus atribut
- `el.dataset.id` → shortcut untuk data-* (modern)

`el.value`

- Khusus untuk elemen form: `<input>`, `<textarea>`, `<select>`
- Baca: `const isi = input.value`
- Kosongkan: `input.value = ""`
- Tidak bisa dipakai pada elemen biasa seperti `<p>` atau `<div>`

classList — Manipulasi Class CSS

`add("nama")`

- Tambah class ke elemen — tidak duplikat jika sudah ada

`remove("nama")`

- Hapus class dari elemen — tidak error jika class tidak ada

`toggle("nama")`

- Tambah jika belum ada, hapus jika sudah ada
- Berguna untuk on/off: dark mode, menu buka/tutup

`contains("nama")`

- Cek apakah class ada — kembalikan true atau false
- Contoh: `if (el.classList.contains("active")) { ... }`

Inline Style — `el.style`

- Menulis style langsung ke atribut style elemen
- Property pakai camelCase: `background-color` → `backgroundColor`
- `el.style.display = "none"` → sembunyikan elemen
- `el.style.display = ""` → kembalikan ke default CSS

1.2 | Read, Update, Create & Hapus Elemen

CRUD elemen DOM — empat operasi dasar manipulasi halaman

```
// BACA konten
el.textContent // teks saja (aman)
el.innerHTML   // termasuk tag HTML
el.value       // nilai input/textarea
el.getAttribute('href') // baca atribut

// UBAH konten
el.textContent = 'Teks baru';
el.innerHTML = '<strong>Bold</strong>';
el.setAttribute('data-id', '42');
el.removeAttribute('disabled');

// UBAH style
el.style.color = 'red'; // inline style
el.classList.add('active'); // tambah class
el.classList.remove('hidden'); // hapus class
el.classList.toggle('dark'); // on/off
el.classList.contains('open'); // cek: true/false
```

```
// BUAT elemen baru
const li = document.createElement('li');
li.textContent = 'Item baru';
li.className = 'todo-item';
li.dataset.id = '99'; // data-id='99'

// TAMBAH ke DOM
parent.append(li); // akhir parent
parent.prepend(li); // awal parent
ref.before(li); // sebelum ref
ref.after(li); // sesudah ref
parent.insertBefore(li, ref);

// HAPUS dari DOM
li.remove();
parent.removeChild(li);

// CLONE elemen
const copy = li.cloneNode(true);
// true = clone beserta children-nya
parent.append(copy);

// PINDAH elemen (otomatis lepas dari parent lama)
otherParent.append(li);
```

SEKSI 2

Events & Interaksi

addEventListener — cara JavaScript / halaman web merespons tindakan pengguna.

2.1 | addEventListener & Jenis Event

Pasang 'action' pada elemen — JS bereaksi saat user berinteraksi

```
// Sintaks dasar
elemen.addEventListener('event', handler);

// Contoh: klik tombol
const btn = document.querySelector('#btn');
btn.addEventListener('click', function(e) {
  console.log('Diklik!', e.target);
});

// Arrow function (lebih ringkas)
btn.addEventListener('click', (e) => {
  e.preventDefault(); // cegah default browser
  e.stopPropagation(); // hentikan bubble ke parent
});

// Hapus listener
const fn = () => console.log('klik');
btn.addEventListener('click', fn);
btn.removeEventListener('click', fn);
```

Mouse

click, dblclick
mouseover, mouseout
mousedown, mouseup
mousemove

Keyboard

keydown, keyup
keypress
Ctrl/Shift/Alt via e.ctrlKey

Form

submit, reset
change, input
focus, blur
select

Window

load, DOMContentLoaded
resize, scroll
online, offline

2.2 | Event Delegation — Satu Listener, Banyak Elemen

Pasang listener di parent, tangkap event dari semua children

```
// TANPA delegation — tidak efisien
// Harus pasang listener ke setiap <li>
document.querySelectorAll('li').forEach(li => {
  li.addEventListener('click', handler); // ✗
});
// Masalah: elemen BARU tidak ter-cover!

// DENGAN Event Delegation — efisien
document.querySelector('#list')
  .addEventListener('click', (e) => {

    // Cek target mana yang diklik
    if (e.target.matches('.delete-btn')) {
      e.target.closest('li').remove();
    }
    if (e.target.matches('.edit-btn')) {
      const id = e.target.dataset.id;
      editItem(id);
    }
  });

// Elemen BARU yang di-append juga ter-cover otomatis!
```

```
// Keyboard Events
document.addEventListener('keydown', (e) => {
  if (e.key === 'Enter') submitForm();
  if (e.key === 'Escape') closeModal();
  if (e.ctrlKey && e.key === 's') save();
});

// Input real-time
const input = document.querySelector('#search');
input.addEventListener('input', (e) => {
  filterList(e.target.value);
});

// Form submit
form.addEventListener('submit', (e) => {
  e.preventDefault(); // cegah reload
  const data = new FormData(form);
  const nama = data.get('nama');
  processData({ nama });
});

// DOMContentLoaded — jalankan JS setelah DOM siap
document.addEventListener('DOMContentLoaded', () => {
  init(); // aman, semua elemen sudah ada
});
```

SEKSI 3

Praktik

Tiga project DOM — dari sederhana hingga timer interaktif

CRUD todo murni JavaScript — tambah, selesaikan, hapus

```
<!-- index.html -->
<div class='app'>
  <input id='inp' placeholder='Tambah todo...'/>
  <button id='add'+ Tambah</button>
  <ul id='list'></ul>
  <p id='count'>0 todo tersisa</p>
</div>
```

```
/* style.css */
li.done span { text-decoration: line-through;
               opacity: 0.4; }
.btn-done { background: #2ecc71; }
.btn-del  { background: #ff556a; }
li { display: flex; gap: 8px;
    align-items: center;
    padding: 8px 12px;
    background: #28283c;
    border-radius: 8px;
    margin-bottom: 6px; }
```

► Cara Menjalankan

```
# Buka index.html dengan Live Server
# Ketik todo → Enter atau klik + Tambah
# Klik ✓ → coret teks (done)
# Klik ✕ → hapus item
# Counter otomatis update
```

```
// app.js
let todos = [];

document.querySelector('#add').addEventListener('click', addTodo);
document.querySelector('#inp').addEventListener('keydown', e => {
  if (e.key === 'Enter') addTodo();
});

function addTodo() {
  const text = document.querySelector('#inp').value.trim();
  if (!text) return;
  todos.push({ id: Date.now(), text, done: false });
  document.querySelector('#inp').value = '';
  render();
}

// Event delegation untuk toggle & delete
document.querySelector('#list').addEventListener('click', e => {
  const id = +e.target.closest('li')?.dataset.id;
  if (e.target.matches('.btn-done'))
    todos = todos.map(t => t.id===id ? {...t,done:!t.done} : t);
  if (e.target.matches('.btn-del'))
    todos = todos.filter(t => t.id !== id);
  render();
});

function render() {
  document.querySelector('#list').innerHTML =
    todos.map(t => `<li data-id='${t.id}'
      class='${t.done?"done":""}'>
      <span>${t.text}</span>
      <button class='btn-done'>✓</button>
      <button class='btn-del'>✕</button>
    </li>`).join('');
  const left = todos.filter(t=>!t.done).length;
  document.querySelector('#count').textContent =
    left + ' todo tersisa';
}
```

PRAKTIK 2 | Galeri Foto + Filter Kategori

PRAKTIK

Filter item tanpa reload — classList show/hide + querySelectorAll

```
<!-- gallery.html -->
<div class='filter-btns'>
  <button class='filter active'
data-cat='all'>Semua</button>
  <button class='filter' data-cat='alam'>Alam</button>
  <button class='filter' data-cat='kota'>Kota</button>
  <button class='filter'
data-cat='kuliner'>Kuliner</button>
</div>
<div class='gallery' id='gallery'>
  <div class='card' data-cat='alam'>
    <img src='alam1.jpg'><p>Gunung Bromo</p>
  </div>
  <div class='card' data-cat='kota'>
    <img src='kota1.jpg'><p>Jakarta Malam</p>
  </div>
  <!-- lebih banyak card -->
</div>
```

• Output

```
// Klik 'Alam' → hanya foto alam tampil
// Klik 'Semua' → semua foto tampil
// Tombol aktif berubah warna
```

```
// gallery.js
const filters = document.querySelectorAll('.filter');
const cards = document.querySelectorAll('.card');

filters.forEach(btn => {
  btn.addEventListener('click', () => {
    // Update tombol aktif
    filters.forEach(b => b.classList.remove('active'));
    btn.classList.add('active');

    const cat = btn.dataset.cat;

    // Tampilkan / sembunyikan card
    cards.forEach(card => {
      const show = cat === 'all' || card.dataset.cat === cat;
      card.classList.toggle('hidden', !show);
    });
  });
});

/* CSS */
.card { transition: opacity 0.3s, transform 0.3s; }
.card.hidden { display: none; }
.filter.active { background: #f7df1e; color: #111; }
```

Note:

- Perbaiki struktur CSS
- Tambahkan lebih banyak card
- Tambahkan lebih banyak button pilihan

PRAKTIK 3 | Countdown Timer + Progress Bar

PRAKTIK

setInterval, manipulasi DOM real-time, CSS progress animasi

```
<!-- timer.html -->
<div class='timer'>
  <div class='display' id='display'>25:00</div>
  <div class='progress-wrap'>
    <div class='bar' id='bar'></div>
  </div>
  <div class='controls'>
    <button id='start'>▶ Start</button>
    <button id='pause'>⏸ Pause</button>
    <button id='reset'>⌛ Reset</button>
  </div>
  <p id='status'>Siap mulai</p>
</div>
```

Note:

- Tambahkan struktur CSS
- Buat tampilan web posisi di tengah centralize

```
// timer.js
const TOTAL = 25 * 60; // 25 menit dalam detik
let remaining = TOTAL;
let interval = null;

function tick() {
  remaining--;
  if (remaining <= 0) { clearInterval(interval); done(); return; }
  updateDisplay();
}

function updateDisplay() {
  const m = String(Math.floor(remaining/60)).padStart(2, '0');
  const s = String(remaining % 60).padStart(2, '0');
  document.querySelector('#display').textContent = m+':'+s;
  const pct = ((TOTAL-remaining)/TOTAL)*100;
  document.querySelector('#bar').style.width = pct+'%';
}

document.querySelector('#start').addEventListener('click', () => {
  if (!interval) interval = setInterval(tick, 1000);
  document.querySelector('#status').textContent = 'Berjalan...';
});

document.querySelector('#pause').addEventListener('click', () => {
  clearInterval(interval); interval = null;
  document.querySelector('#status').textContent = 'Dijeda';
});

document.querySelector('#reset').addEventListener('click', () => {
  clearInterval(interval); interval = null;
  remaining = TOTAL; updateDisplay();
});
```

Buatlah sebuah halaman web sederhana yang memiliki:

- input bertipe angka (`type="number"`)
- tombol "Tampilkan"
- area untuk menampilkan keterangan hasil

Ketentuan

- Saat tombol "Tampilkan" diklik, tampilkan keterangan: "Kamu memasukkan angka: [angka yang diinput]"
- Saat user menekan tombol Enter di keyboard, aksi yang sama terpicu.
- Setelah tombol diklik, input dikosongkan kembali
-

Konsep yang harus digunakan

- `document.querySelector()` – untuk seleksi elemen
- `addEventListener('click')` – untuk event klik tombol
- `addEventListener('keydown')` – untuk event keyboard
- `element.textContent` – untuk mengubah isi teks
- `classList.remove()` – untuk menampilkan elemen yang tersembunyi